⚡ MEDULLA • MEDICAL LEARNING PLATFORM

System Architecture, Tech Stack & API Specification

A complete technical guide covering the frontend design system, Node.js backend infrastructure, hybrid database model, Google Gemini AI integration, Razorpay payments, and all REST API endpoints.

APPLICATION
Medulla (MBBS, NEET PG & USMLE Platform)

ARCHITECTURE
React 18 SPA + Node.js 22 Native ESM Micro-Core

VERSION
Production Release v2.4

DATE GENERATED
October 2026

1. High-Level Architecture Overview

Medulla is engineered as a high-performance, resilient web application designed specifically for medical students. It combines intense interactive gamification (1v1 Duels with Elo ratings), structured revision (over 11,000+ official PYQs), social peer collaboration (Twitter-style community threads), and generative AI evaluation (Oral Viva and diagnostic clinical simulations).

Client Tier (SPA)

React 18 single-page application built on top of Vite 5. Features Radix UI headless components, Framer Motion transitions, Tailwind CSS responsive layouts with full dark/light theme switching, and client-side route dispatching.

Backend Tier (Node.js)

Zero-overhead, pure Node.js 22 native ESM server. Implements custom low-latency HTTP request routing, PBKDF2 cryptography, rate-limiting token buckets, real-time Elo match resolution, and secure cookie management.

Hybrid Data Persistence

Optimistic state store with atomic revision snapshot locking. Local database persistence in `runtime-data/users.json` with seamless cloud replication to Supabase PostgreSQL and private S3-compatible asset bucket storage.

AI & External Integrations

Deep integration with Google Gemini LLMs (`gemini-3.5-flash-lite`) for real-time oral viva simulations and clinical reasoning scoring, paired with Razorpay checkout for annual subscriptions and access tiers.

2. Core Technology Stack

DOMAIN	TECHNOLOGY	DESCRIPTION & KEY USAGE
Frontend Framework	React 18.3.1	Declarative component architecture, custom hooks, portals
Bundler & Dev Server	Vite 5.4.10	Lightning-fast HMR, Rollup chunking, asset pre-bundling
Styling & Themes	Tailwind CSS 3.4.17 + PostCSS	Utility-first styles, dark/light theme switching, animated keyframes
UI Component Primitives	Radix UI Primitive Suite	WAI-ARIA compliant Dialogs, Tooltips, Dropdowns, Progress bars
Animation Engine	Framer Motion 12.42.0	Smooth page view transitions, duel arena animations, micro-interactions
Iconography	Lucide React 1.21.0	Comprehensive medical, navigation, and gaming icon system
Forms & Validation	React Hook Form 7.80 + Zod 4.4	Performant input state, type-safe schema validation

DOMAIN	TECHNOLOGY	DESCRIPTION & KEY USAGE
Server Environment	Node.js 22.x (Native ESM)	Custom router with `node:http`, zero bloated frameworks
Security & Auth	PBKDF2 (sha512, 220k iter)	Timing-safe equal authentication, HttpOnly signed sessions
AI Engine	Google Gemini API	Oral viva voice/text grading, clinical cases, short notes review
Payments	Razorpay Node SDK 2.9.8	UPI, Cards, and Netbanking order management and webhook verifications
Database / Storage	Atomic State Store / Supabase	Snapshot serialization with optimistic CAS and Supabase PostgreSQL / S3

3. Dependencies & Libraries Catalog

All dependencies configured in `package.json` and their designated responsibilities in the Medulla platform:

Production Dependencies

PACKAGE NAME	VERSION	PURPOSE IN MEDULLA
`react`	^18.3.1	Core UI rendering engine and declarative component hierarchy.
`react-dom`	^18.3.1	DOM rendering, modal overlays, portals, and event dispatching.
`framer-motion`	^12.42.0	Page transitions, 1v1 battle animations, toast alerts, modal fades.
`lucide-react`	^1.21.0	Crisp SVG iconography across clinical, community, and practice interfaces.
`@radix-ui/react-dialog`	^1.1.17	Accessible modal dialogs (Search `Cmd+K`, Circle details, Confirmation modals).
`@radix-ui/react-dropdown-menu`	^2.1.18	Account user-chip menus, notifications popover, thread action menus.
`@radix-ui/react-tabs`	^1.1.15	Tabbed panels for Communities, Practice catalogs, and Review modes.
`@radix-ui/react-tooltip`	^1.2.10	Contextual hover tooltips on navigation links and header actions.
`@radix-ui/react-progress`	^1.1.10	Visual progress tracking for question goals and timed battle countdowns.
`@radix-ui/react-switch`	^1.3.1	Study settings toggles (daily reminders, reduced motion, notifications).
`@tanstack/react-query`	^5.101.2	Data fetching, caching, background synchronization, and optimistic updates.
`tailwindcss`	^3.4.17	Utility styling framework supporting responsive layouts and custom themes.
`tailwindcss-animate`	^1.0.7	Plugin providing entrance, exit, and pulse keyframe animations.

PACKAGE NAME	VERSION	PURPOSE IN MEDULLA
`clsx`	^2.1.1	Lightweight conditional class name concatenation utility.
`tailwind-merge`	^3.6.0	Merges Tailwind classes safely without style specificity collisions.
`class-variance-authority`	^0.7.1	Provides declarative variant definitions for buttons, badges, and cards.
`zod`	^4.4.3	TypeScript-first schema declaration and runtime data validation.
`react-hook-form`	^7.80.0	Uncontrolled performant form handling for user profiles and feedback.
`@hookform/resolvers`	^5.4.0	Integrates Zod validation schemas seamlessly with React Hook Form.
`razorpay`	^2.9.8	Official Razorpay Node.js SDK for checkout order creation and verification.
`postcss`	^8.5.16	CSS processor required by Tailwind CSS.
`autoprefixer`	^10.5.2	Adds vendor prefixes automatically to ensure cross-browser compatibility.
`typescript`	^6.0.3	Static type definitions and TypeScript compilation support.

4. Complete REST API Specification

The backend routes all requests via `server.mjs`. All payloads and responses are formatted as UTF-8 JSON.

4.1 Authentication & User Sessions

METHOD	ENDPOINT	DESCRIPTION & BEHAVIOR
POST	`/api/auth/signup`	Creates a new learner account. Validates unique email and mobile, derives PBKDF2 hash, issues session cookie.
POST	`/api/auth/login`	Verifies credentials with timing-safe hash comparison. Automatically upgrades legacy password hashes.
POST	`/api/auth/logout`	Invalidates the active session token in memory and clears the client session cookie.
GET	`/api/auth/session`	Returns active user profile, subscription status, and token balances if authenticated.

4.2 User Profile & Preferences

METHOD	ENDPOINT	DESCRIPTION & BEHAVIOR
GET	`/api/users/search`	Searches learners by name, username, or medical college for peer connections.
GET	`/api/users/:userId`	Retrieves a sanitized public learner profile (XP rating, streak, medical college, stats).
PATCH	`/api/profile`	Updates username, medical college, contact number, or uploads a new base64 profile photo.
PATCH	`/api/profile/stats`	Syncs practice accuracy, attempted questions, streak counters, and performance history.
PATCH	`/api/profile/question-bookmarks`	Toggles a question in the student's personal revision bookmark repository.
GET	`/api/tokens`	Retrieves the student's daily token allowance, remaining balance, and next reset timestamp.

4.3 Community Study Circles & Threads

METHOD	ENDPOINT	DESCRIPTION & BEHAVIOR
GET	`/api/communities`	Lists all public study circles, member counts, topics, and membership statuses.
POST	`/api/communities`	Creates a new study circle with name, subject topic, and description (requires Ultra/Premium).
POST	`/api/communities/:id/join`	Adds the authenticated user to the specified study circle.

METHOD	ENDPOINT	DESCRIPTION & BEHAVIOR
POST	`/api/communities/:id/messages`	Publishes a new discussion thread or nested reply with optional image attachments.
DELETE	`/api/communities/:id/members/:mId`	Removes a member from the circle (restricted to the circle administrator).

4.4 Direct Messaging & Social Peer Network

METHOD	ENDPOINT	DESCRIPTION & BEHAVIOR
GET	`/api/direct-messages`	Retrieves all 1-on-1 conversations and the most recent message for the active user.
POST	`/api/direct-messages/open`	Finds or opens a private conversation channel with another medical student.
POST	`/api/direct-messages/:id/messages`	Sends a private text or image message within a direct conversation.
GET	`/api/friends`	Lists active friends and pending incoming/outgoing friendship invitations.
POST	`/api/friends`	Dispatches a new peer friend request to a target learner.

4.5 1v1 Competitive Duels & Matchmaking Engine

METHOD	ENDPOINT	DESCRIPTION & BEHAVIOR
GET	`/api/duels/questions`	Returns 5 balanced competitive MCQs for an active 1v1 battle session.
POST	`/api/duels/answer`	Locks in an answer with timestamp validation to compute speed and accuracy points.
GET	`/api/duels/status`	Polls the live duel state, question indices, and opponent score in real-time.
POST	`/api/duels/complete`	Finalizes the duel, updates Elo K-32 ratings, records win/loss, and awards XP.
POST	`/api/duels/challenge/create`	Issues a direct 1v1 battle challenge to a friend.
GET	`/api/duels/challenge/status`	Monitors whether an outgoing challenge was accepted, declined, or timed out.
POST	`/api/duels/challenge/accept`	Accepts an incoming duel challenge and initiates the arena room.
POST	`/api/duels/challenge/decline`	Declines an incoming duel invitation.
POST	`/api/duels/rated/queue`	Joins the global competitive ranked matchmaking queue.
GET	`/api/duels/rated/queue`	Polls matchmaking queue status to detect paired opponent.
DELETE	`/api/duels/rated/queue`	Cancels waiting and withdraws from the matchmaking queue.

4.6 Practice Library & Learning Analytics

METHOD	ENDPOINT	DESCRIPTION & BEHAVIOR
GET	/api/practice	Delivers question library catalogs across NEET PG, INI-CET, FMGE, and USMLE Step 1.
POST	/api/practice/attempts	Logs question attempts, records correct/incorrect flags, subject ID, and solving time.
GET	/api/dashboard	Returns analytics summary: daily study goal, accuracy trends, and weak subject areas.
PATCH	/api/dashboard/goal	Updates the student's daily target question volume (20, 40, 60, or 100).
GET	/api/leaderboard	Returns top learner rankings filtered by National or State/College criteria.

4.7 AI Oral Viva, Clinical Cases & Short Notes (Google Gemini)

METHOD	ENDPOINT	DESCRIPTION & BEHAVIOR
POST	`/api/generate-question`	Generates a clinically framed medical MCQ using Google Gemini LLM.
POST	`/api/viva/sessions`	Starts an oral viva examination session for a selected subject and chapter.
POST	`/api/viva/sessions/:id/answers`	Submits student oral answer for AI examiner evaluation, grade score, and clinical tips.
POST	`/api/viva/sessions/:id/advance`	Advances the viva session to subsequent follow-up oral inquiries.
POST	`/api/clinical-cases/sessions`	Generates an interactive patient case scenario with history and vital signs.
GET	`/api/clinical-cases/sessions/:id`	Fetches active case progression, lab investigations, and differential notes.
POST	`/api/clinical-cases/sessions/:id/answers`	Evaluates student differential diagnosis and management plan against standard clinical protocols.
ALL	`/api/short-notes/*`	Handles AI assessment of university exam theory answers and short note submissions.

4.8 Monetization, Reviews & Operational Health

METHOD	ENDPOINT	DESCRIPTION & BEHAVIOR
POST	`/api/create-order`	Creates a signed Razorpay payment order for Lite (₹499), Ultra (₹999), or Premium (₹1499).
POST	`/api/verify-payment`	Cryptographically verifies the HMAC SHA256 payment signature and unlocks practice access.
GET	`/api/reviews`	Delivers verified student reviews, ratings distribution, and testimonials.
POST	`/api/reviews`	Submits learner feedback and 1-5 star ratings.
GET	`/api/summary`	Delivers global platform counters (total active users, questions solved, study groups).
GET	`/api/storage/status`	Inspects local/cloud storage health, revision count, and upload bucket readiness.
GET	`/health/live`	Liveness probe returning HTTP 200 for container orchestrators.
GET	`/health/ready`	Readiness probe confirming database and state store initialization.

5. Data Repositories & Storage Architecture

Question Bank Storage

- `data/practice-question-bank.json` : 11,102+ verified official PYQs (NEET PG 2017–2024, INI-CET, FMGE, USMLE).
- `data/topic-wise-question-bank.json` : Curated medical curriculum across 19 MBBS subjects.
- `src/data/medicalColleges.js` : Standardized directory of recognized Indian medical institutions and states.
- `src/data/paymentPlans.js` : Tiered pricing configurations with annual feature matrix.

Media & User Uploads Handling

User avatars and thread attachments are validated for MIME type (PNG, JPEG, WebP, GIF) and size limits (5MB). Uploaded files are cryptographically named, stored locally in `runtime-data/uploads`, and synced to Supabase storage buckets in production deployments.